

모듈별 개발 지침 및 작업 기준

G2B 유사제품 검색·견적 자동화 프로젝트

이 문서는 프로젝트를 구성하는 Python 백엔드, Svelte 프론트엔드, Electron 견적 프로그램의 개발 원칙과 작업 완료 기준을 한 문서로 정리한 것이다.

문서 목적

- 새 작업을 시작할 때 수정 위치를 빠르게 찾는다.
- 데이터 출처와 결과 재현성을 훼손하지 않는다.
- 모듈 경계를 지키면서 필요한 범위만 변경한다.
- 테스트 통과뿐 아니라 실제 사용자 동작까지 확인한다.

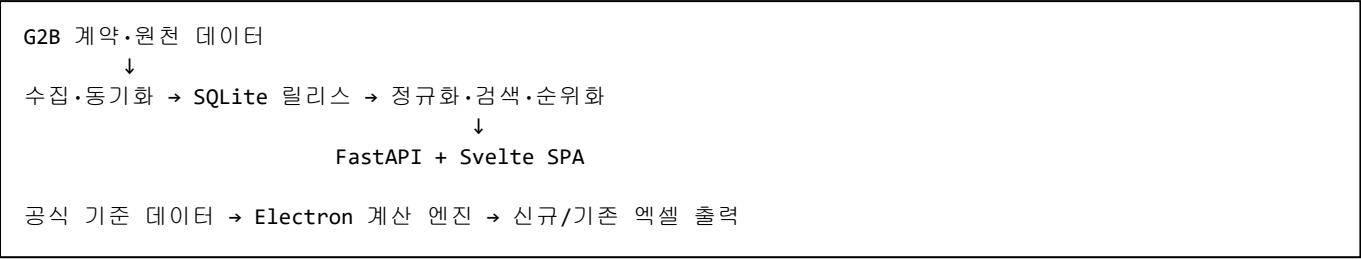
적용 대상

구분	주요 경로	역할
Python 서비스	src/g2b_compare/	수집, 검색, 비교, API
웹 화면	frontend/	카탈로그와 견적 SPA
데스크톱 앱	electron-estimator/	견적 계산과 엑셀 출력
검증 코드	tests/, 각 패키지 tests/	회귀 및 인수 검증

핵심 원칙: 출처가 확인되는 데이터만 사용하고, 실패한 작업은 마지막 정상 상태를 훼손하지 않으며, 동일 입력은 동일 결과를 만들어야 한다.

1. 전체 구조와 공통 작업 기준

시스템 구성



Python 서비스와 Electron 프로그램은 같은 저장소에 있지만 독립된 제품이다.
Python 서비스의 빌드 성공만으로 Electron 변경이 검증된 것은 아니며, 반대로 같다.

공통 개발 원칙

- 1. 작은 변경: 현재 요구사항을 해결하는 최소 범위만 수정한다.
- 2. 출처 보존: 원본 응답, 스냅샷, 릴리스, 산출물의 식별자를 구분한다.
- 3. 결정론 유지: 정렬, 반올림, 식별자, 직렬화 순서를 명시적으로 관리한다.
- 4. 경계 검증: 사용자 입력, 외부 API, 파일, IPC는 진입 시점에 파싱한다.
- 5. 실패 폐쇄: 검증이 불완전하면 이전 정상 상태를 유지하고 공개하지 않는다.
- 6. 비밀 보호: API 키, 원문 응답, 로컬 경로, 스택 추적을 로그나 화면에 노출하지 않는다.

작업 순서

단계	수행 내용	완료 증거
조사	대상 코드와 호출 관계, 관련 테스트 확인	수정 범위 목록
구현	기존 구조와 스타일에 맞춘 최소 변경	제한된 diff
정적 검증	포맷, 린트, 타입 검사	명령 종료 코드 0
동작 검증	관련 테스트와 실제 진입점 실행	실제 출력 또는 화면
정리	생성 파일과 프로세스 정리	깨끗한 작업 결과

공통 금지 사항

- 생성된 번들, dist/, ASAR 내부 파일, 적용 완료된 SQL 마이그레이션을 직접 편집하지 않는다.
- 실패하는 테스트를 삭제·건너뛰기·악화하여 통과시키지 않는다.
- 원본 엑셀 파일을 덮어쓰거나 외부에서 가져온 수식을 실행하지 않는다.
- 부분 동기화 또는 부분 빌드를 마지막 정상 릴리스 위에 공개하지 않는다.

2. Python 백엔드 개발 지침

담당 범위

src/g2b_compare/는 G2B 계약 확인, 원천 데이터 수집, 동기화, SQLite 저장, 정규화, 검색·순위화, 견적 서비스와 FastAPI 진입점을 담당한다.

작업	위치
API 명세·쿼터	contracts/
HTTP 전송·응답 봉투	sources/
페이지·체크포인트·공개	sync/
검색·비교 계산	search/, ranking/
유스케이스	services/
CLI와 운영 상태	observability/
HTTP/SPA 제공	web/

구현 기준

- Python 3.12와 3.13을 지원한다.
- 외부 입력은 Pydantic 모델로 파싱하고, 내부 값 객체는 기본적으로 불변으로 둔다.
- SQLite 행을 공개 경계로 전달하지 않고 타입이 지정된 모델로 변환한다.
- 기본 검색·비교 경로는 로컬 DB와 인덱스만 읽는다.
- 네트워크 호출은 계약 캡처, 동기화, 크롤링, 명시적 라이브 진단에만 둔다.
- 공급자 오류는 안정적인 내부 오류 코드로 변환하고 비밀 정보를 제거한다.

작업 시 확인할 것

1. 변경하려는 심볼의 호출자와 결과 소비자를 확인한다.
2. 검색 후보군과 순위 계산을 구분한다.
3. 식별자와 정렬 순서가 기존 결과를 불필요하게 바꾸지 않는지 확인한다.
4. 동기화 실패 시 현재 활성 릴리스가 그대로 유지되는지 확인한다.
5. CLI 또는 API 변경이면 실제 명령이나 요청을 실행한다.

검증 명령

```
uv run ruff format --check .
uv run ruff check .
uv run basedpyright
uv run pytest -q tests/<관련영역>
uv run g2b-compare --help
```

전체 영향을 주는 변경은 마지막에 `uv run pytest -q`까지 실행한다.

3. 데이터베이스·릴리스 개발 지침

담당 범위

src/g2b_compare/db/는 SQLite 연결 정책, 마이그레이션, 수집 출처, 원문 보관, 스냅샷과 활성 릴리스의 일관성을 관리한다.

핵심 불변 조건

- 적용된 마이그레이션은 체크섬으로 잠기며 수정하지 않는다.
- 스키마 변경은 새로운 순방향 마이그레이션으로 추가한다.
- 공개된 스냅샷과 릴리스 구성요소는 제자리에서 수정하지 않는다.
- 릴리스는 카탈로그·정규화·인덱스 등 정확한 구성 세대를 고정한다.
- 원문 본문은 내용 기반 SHA로 식별하고 참조 중인 데이터는 삭제하지 않는다.
- 여러 테이블의 상태를 함께 바꾸는 작업은 명시적 트랜잭션으로 처리한다.

변경 유형별 기준

변경	필수 확인
마이그레이션 추가	신규 DB와 기존 DB 모두 적용
저장소 로직	입력 반복 시 중복 생성 없음
릴리스 공개	검증 실패 시 이전 포인터 유지
원문 정리	참조된 SHA 보존
잠금 처리	시간 지연이 아닌 정확한 상태로 동기화

테스트 작성 기준

- 개발용 .g2b DB를 테스트에서 읽거나 수정하지 않는다.
- tmp_path에 DB를 만들고 실제 마이그레이션 함수를 실행한다.
- 저장소를 우회해 테스트 전용 스키마를 손으로 만들지 않는다.
- 원자성을 검증할 때 정상 경로뿐 아니라 중간 실패도 주입한다.
- 락과 비동기 테스트에 고정 sleep을 사용하지 않는다.

검증 명령

```
uv run pytest -q tests/db
uv run pytest -q tests/materialize tests/sync
uv run ruff check src/g2b_compare/db tests/db
uv run basedpyright
```

4. 평가·품질 증거 개발 지침

담당 범위

src/g2b_compare/evaluation/은 외부에서 작성된 정답 데이터와 예측 결과의 결합을 검증하고, 결정론적 지표와 출시 기준을 계산한다.

신뢰 경계

형식	의미
e0-v1	외부 작성 정답과 해시·개수·계층 정보
e0-export-v1	라벨 없는 평가 대상 내보내기
e0-strict-v1	원천 내보내기와 명시적으로 결합된 외부 정답

구현 기준

- 점수와 임계값은 Decimal로 계산하고 반올림 규칙을 유지한다.
- 지표 계산 전에 SHA-256, 파일 존재, 행 개수, 계층, 풀 식별자를 검증한다.
- 선언된 파일 경로는 안전한 상대 경로만 허용한다.
- 중복 키나 평가 풀 밖의 예측을 조용히 제거하지 않는다.
- 엄격 평가는 실제 예측 산출물을 소비하며 숨겨진 모델 실행을 포함하지 않는다.
- 보류 데이터와 출시 임계값을 편의를 위해 조정하지 않는다.

금지 사항

- 외부 정답을 추론·수정·재라벨링·재정렬하지 않는다.
- 합성 fixture 결과를 실제 품질 증거로 주장하지 않는다.
- 학습 또는 튜닝 행을 보류 평가에 섞지 않는다.
- 정확한 기준에 임의의 epsilon 허용 오차를 추가하지 않는다.
- 검증 실패를 경고로 바꾸어 계속 진행하지 않는다.

완료 기준

```
uv run pytest -q tests/evaluation tests/unit/test_e0_schema.py
uv run basedpyright
```

평가 산출물에는 원천 export와 materialization 식별자가 남아 있어야 하며, 다른 환경에서 같은 입력으로 동일한 정렬과 지표가 재현되어야 한다.

5. FastAPI·웹 제공 계층 개발 지침

담당 범위

src/g2b_compare/web/는 API, 상태 점검, 레거시 화면, Svelte SPA 정적 파일과 서비스 워커를 하나의 FastAPI 앱으로 조합한다.

라우팅 기준

- /, /data, /estimates, 견적 상세 경로만 SPA HTML로 연결한다.
- /api/*, export, health, assets, /live, /priority, /sync는 SPA fallback 대상이 아니다.
- /healthz, /livez는 프로세스 생존을 나타낸다.
- /readyz는 카탈로그 데이터 준비 상태를 반영한다.
- 라이브 공급자 조회는 명시적 진단 경로이며 기본 검색 경로가 아니다.

구현 기준

- 라우터는 HTTP 입력·출력 변환만 하고 유스케이스는 서비스에 둔다.
- SQL은 기존 store/reader 경계에서 실행한다.
- API 모델과 상태 코드, 오류 응답 형식을 안정적으로 유지한다.
- 공급자 오류 원문, 키, 로컬 파일 경로를 응답에 포함하지 않는다.
- 근거가 없는 상품 상세 URL을 조합하지 않는다.

SPA 번들 기준

frontend/에서 빌드하면 src/g2b_compare/web/frontend_dist/가 갱신된다. 이 디렉터리는 생성 결과이므로 직접 편집하지 않는다. 앱은 production SPA의 index, worker, assets 또는 부팅 자산이 없으면 의도적으로 실패해야 한다.

검증 순서

```
cd frontend
npm ci
npm test -- --run
npm run build

cd ..
uv run pytest -q tests/web
uv run basedpyright
```

마지막으로 실제 서버를 실행해 정상 페이지, 잘못된 URL, API 응답, 서비스 워커의 콘텐츠 타입을 확인한다.

6. Svelte 프론트엔드 개발 지침

담당 범위

frontend/는 카탈로그 조회, 견적 목록·편집, 데이터 상태를 제공하는 Svelte 5 SPA이다. IndexedDB를 사용해 오프라인 편집과 동기화 대기를 지원한다.

상태·동기화 기준

- 컴포넌트는 Svelte 5 rune을 사용하고 화면 데이터와 동작은 props로 전달한다.
- IndexedDB v1의 store는 catalog_cache, estimates, app_state 세 개다.
- 로컬 견적은 최신 쓰기가 우선이며, 오래된 동기화 완료가 새 편집을 지우면 안 된다.
- 서버에 한 번도 올라가지 않은 빈 초안은 로컬에서 제거할 수 있다.
- 이미 동기화된 문서 삭제는 pending tombstone으로 유지한다.
- syncPendingEstimates는 실행을 직렬화하고 문서별 실패를 안정적으로 기록한다.
- SSE 저장·삭제 이벤트가 오면 현재 화면의 견적 데이터를 무효화한다.

UI 기준

- 루트 DESIGN.md의 토큰, 배치, 접근성 규칙을 따른다.
- CJK 시스템 글꼴, 명확한 focus, 의미 있는 landmark를 유지한다.
- 주요 조작 영역은 최소 44px 높이를 확보한다.
- 표와 금액 데이터의 비교 가능성을 장식보다 우선한다.
- modal과 tooltip은 Escape, 키보드 이동, focus 복귀를 지원한다.
- /live, /priority, /sync 링크를 SPA가 가로채지 않는다.

검증 명령

```
npm ci
npm test -- --run
npm run build
npm run dev
```

IndexedDB 테스트는 fake-indexeddb를 사용한다. UI 변경은 브라우저에서 데스크톱과 좁은 화면을 직접 확인하고, 새로고침·오프라인·재연결 동작도 점검한다.

7. Electron 견적 프로그램 개발 지침

담당 범위

electron-estimator/는 공식 기준 데이터로 견적을 계산하고, 신규 워크북 또는 기존 양식 호환 워크북을 안전하게 생성하는 독립 프로그램이다.

영역	역할
src/domain/	금액, 견적, 출처, 검증 규칙
src/official/	공식 데이터 검증과 선택
src/native/	신규 워크북 생성
src/legacy/	기존 OOXML 분석·패치·출력
src/main/	앱 시작, 프로토콜, IPC, 권한
src/preload/	제한된 renderer bridge
src/renderer/	사용자 작업 화면

구현 기준

- TypeScript strict 옵션과 noUncheckedIndexedAccess, exact optional type을 지킨다.
- IPC, 파일, manifest, 공식 데이터는 Zod로 파싱한다.
- 공식 데이터 검증이 실패하면 첫 BrowserWindow를 열지 않는다.
- 계산 도메인은 Electron과 renderer에 의존하지 않는 순수 로직으로 유지한다.
- renderer는 Node I/O나 Electron 권한 API를 직접 import하지 않는다.
- preload는 검증된 고정 bridge만 노출한다.
- DESIGN.md와 src/renderer/design-contract.ts의 의미를 일치시킨다.

생성·패키징 기준

- npm run build가 dist/main, dist/preload, dist/renderer를 재생성한다.
- dist의 정확한 목록은 scripts/assert-build.mjs가 판정한다.
- ASAR에는 테스트, TypeScript, source map, 데이터셋, 캐시, 비밀을 포함하지 않는다.
- 생성된 dist/, release/, 검증 보고서를 직접 수정하지 않는다.

기본 검증

```
npm run typecheck
npm run build
npm run test:unit
npm run test:integration
npm run test:security
```


8. Electron 보안·레거시 엑셀 작업 기준

Main process 보안

- privileged app scheme을 readiness 이전에 등록한다.
- origin은 정확히 app://app만 사용한다.
- sandbox와 context isolation을 켜고 Node integration과 webview를 끈다.
- navigation, 새 창, download, permission 요청을 기본 거부한다.
- IPC 요청·응답을 채널별 Zod schema로 모두 검증한다.
- live main window, webContents, frame origin, process/routing identity를 확인한다.
- 파일 선택 권한은 작업·frame에 묶인 일회성 120초 capability로 발급한다.

레거시 워크북 처리

검사 → 패치 계획 → OOXML 패치 → 검증 보고서
→ 내구성 임시 저장 → 원자적 공개 → 정리

- XLSX를 신뢰하지 않는 ZIP/XML 패키지로 취급한다.
- 원본 workbook SHA를 고정하고 출력 전후에 다시 확인한다.
- 선택된 manifest와 명시된 cell 범위만 수정한다.
- source workbook은 절대 덮어쓰지 않는다.
- workbook과 validation sidecar를 하나의 검증된 쌍으로 만든다.
- transaction journal은 destination 밖에 두고 새 export 전 복구한다.
- 실패하면 부분 산출물을 제거하고 cleanup receipt를 남긴다.
- journal이 남은 상태에서 성공을 반환하지 않는다.

보안 검증

```
npm run test:security
npm run test:integration
npm run test:e2e
```

E2E 편의를 위해 sender 검증이나 capability 검증을 약화하지 않는다. 테스트는 악성 ZIP, 원본 변조, 중간 crash, underfill, 재시작 복구를 포함해야 한다.

9. 테스트·완료 기준 체크리스트

테스트 작성 기준

- 테스트 데이터와 DB, 워크북, 산출물은 테스트 전용 임시 경로에 만든다.
- 비동기 동작은 정확한 이벤트를 먼저 구독한 뒤 동작을 발생시킨다.
- 시간 기반 sleep이나 운에 따른 polling으로 성공을 만들지 않는다.
- mock은 검증 대상 계약을 유지하며, 핵심 통합 경계를 지워버리지 않는다.
- 실패 테스트를 skip, todo, xfail, .only로 우회하지 않는다.
- 자식 프로세스를 실행한 테스트는 프로세스 트리화 임시 파일을 정리한다.

변경 완료 체크리스트

확인 항목	완료
수정 심볼과 호출 관계를 확인했는가	☐
관련 회귀 테스트가 실패→성공을 증명하는가	☐
포맷·린트·타입 검사가 통과했는가	☐
관련 테스트가 한 번에 안정적으로 통과하는가	☐
실제 CLI/API/UI/워크북 동작을 사용해 보았는가	☐
비밀, 원문, 절대 경로가 노출되지 않는가	☐
생성 파일과 프로세스가 정리되었는가	☐
사용자 작업과 무관한 파일을 수정하지 않았는가	☐

최종 명령

```
# Python 전체
uv run ruff format --check .
uv run ruff check .
uv run basedpyright
uv run pytest -q

# Frontend
cd frontend
npm test -- --run
npm run build

# Electron
cd ../electron-estimator
npm run verify:all
```

verify:all은 typecheck, build, unit, integration, security, 데이터 계약, Electron E2E, renderer, ASAR, artifact oracle, cleanup audit의 11단계를 수행한다.

❗ 완료의 기준은 “코드상으로 될 것 같다”가 아니라 사용자가 접하는 실제 경로에서 요구 동작을 확인한 상태다.